

Hands-On Lab: Modular Payment Processing System with Interfaces and Dependency Injection

Objective

In this lab, you will:

1. Create an interface for a payment processing system.
 2. Implement the interface for multiple payment methods (e.g., Credit Card, PayPal).
 3. Use dependency injection to manage the dependencies of the payment processors.
 4. Write a console application to simulate the payment process.
-

Prerequisites

- Basic understanding of C# interfaces.
 - Familiarity with Dependency Injection (DI) concepts.
 - A development environment with .NET installed.
-

Lab Instructions

Step 1: Create the Payment Processor Interface

Define an interface, `IPaymentProcessor`, that will serve as a contract for all payment processors.

```
public interface IPaymentProcessor
{
    void ProcessPayment(decimal amount);
}
```

Step 2: Implement Payment Processors

Create two classes, `CreditCardProcessor` and `PayPalProcessor`, that implement the `IPaymentProcessor` interface.

CreditCardProcessor

```
public class CreditCardProcessor : IPaymentProcessor
{
    public void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing credit card payment of {amount:C}");
    }
}
```

PayPalProcessor

```
public class PayPalProcessor : IPaymentProcessor
{
    public void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing PayPal payment of {amount:C}");
    }
}
```

Step 3: Set Up Dependency Injection

Install the `Microsoft.Extensions.DependencyInjection` package if not already included in your project.

In the `Program.cs` file, configure the DI container to inject the required dependencies.

```
using Microsoft.Extensions.DependencyInjection;

class Program
{
    static void Main(string[] args)
    {
        // Set up the DI container
        var services = new ServiceCollection();
        services.AddScoped<IPaymentProcessor, CreditCardProcessor>(); // Switch to
        PayPalProcessor as needed

        var serviceProvider = services.BuildServiceProvider();

        // Resolve the payment processor
        var paymentProcessor = serviceProvider.GetRequiredService<IPaymentProcessor>();

        // Simulate a payment
        paymentProcessor.ProcessPayment(123.45M);
    }
}
```

Step 4: Test the Application

1. Run the application.
2. Observe the console output to verify the payment processor is functioning as expected.
3. Modify the `services.AddScoped<IPaymentProcessor, ...>` line to switch between `CreditCardProcessor` and `PayPalProcessor`.

Step 5: Extend the Lab (Optional)

- Add a new payment method, such as `BankTransferProcessor`, implementing the `IPaymentProcessor` interface.
- Enhance the interface to include additional methods like `ValidatePaymentDetails`.
- Create a user input mechanism to dynamically select the payment method at runtime.

Deliverables

- A working console application that demonstrates modular payment processing with DI.
- A write-up or screenshots of the console outputs for different payment methods.

This exercise teaches practical usage of interfaces and DI while highlighting the benefits of loosely coupled designs.